

Production AI Capstone

Combine requirements, data, models, retrieval, tools, evaluation, deployment, and documentation into one defensible system.



WHY THIS MATTERS

Start from what you already know

All prior weeks

- 01 Explain requirements and architecture in plain language.
- 02 Use evidence-first implementation in a small worked example.
- 03 Complete the guided lab and verify security cost and deployment.

CORE CONCEPT

The vocabulary of this topic

Combine requirements, data, models, retrieval, tools, evaluation, deployment, and documentation into one defensible system.

requirements and architecture: Requirements define testable needs and constraints; architecture assigns them to components and interfaces.

evidence-first implementation: Evidence-first implementation defines observable acceptance checks before building a successful demo path.

evaluation and observability: Evaluation measures expected behavior; observability records live metrics, logs, and traces.

security cost and deployment: Production release constrains permissions, secrets, resources, budgets, health, and rollback.

FOUR CONNECTED IDEAS

How the concept works

01

**requirements and
architecture**



02

**evidence-first
implementation**



03

**evaluation and
observability**



04

**security cost and
deployment**

User need → explicit contract → bounded components → evidence and evaluation → monitored deployment.

CORE CONCEPT

Read every part of the mechanism

User need → explicit contract → bounded components → evidence and evaluation → monitored deployment.

requirement: A testable user need and constraint.

architecture: Components and data flow.

evidence: Observable proof of behavior.

WORKED EXAMPLE

Worked example by hand

Turn “answer questions about reports” into ingestion, retrieval, citation, refusal, latency, and access-control requirements.

01

Known values

02

Apply the mechanism

03

Interpret the result

RUN IT AND INSPECT THE RESULT

Map the concept to Python

```
acceptance = {  
    "citation_support": 0.95,  
    "retrieval_recall_at_5": 0.85,  
    "p95_latency_seconds": 5,  
    "monthly_budget_usd": 10,  
}  
print(acceptance)
```

OUTPUT

```
{'citation_support': 0.95, 'retrieval_recall_at_5': 0.85,  
'p95_latency_seconds': 5, 'monthly_budget_usd': 10}
```

Each line corresponds to a concept already explained.

QUALITY GATE

Build the complete mental model

- ☒ Translate the user need into measurable quality, safety, latency, and cost acceptance criteria.

- ☒ Draw data flow and trust boundaries before selecting frameworks or implementation details.

- ☒ Demonstrate unsupported, malformed, denied, timeout, and insufficient-evidence cases.

- ☒ Release with versioned artifacts, monitoring, budget alerts, runbook, limitations, and rollback.

FOUR CONNECTED IDEAS

Notebook readiness map

01

**requirements and
architecture**



02

**evidence-first
implementation**



03

**evaluation and
observability**



04

**security cost and
deployment**

Use the concepts from this lesson to read and complete
<https://github.com/curiously/Al-Bootcamp/blob/master/19.langgraph-crypto-agents.ipynb>.

RISK REVIEW

Common beginner mistakes

01 Starting implementation before defining acceptance criteria.

02 Demonstrating one successful path while hiding failure, security, or cost behavior.

GUIDED LAB

Build and demonstrate a small deployed AI system with documented requirements, tests, traces, and limitations.

01

02

03

04

Predict

Run

Inspect

Explain

SESSION SYNTHESIS

Keep the system, not isolated definitions.

- 01 Explain requirements and architecture in plain language.
- 02 Use evidence-first implementation in a small worked example.
- 03 Complete the guided lab and verify security cost and deployment.

NEXT EVIDENCE

A deployed capstone, source repository, architecture diagram, evaluation report, demo, runbook, and limitations statement.